

TASKS 1–2

Web Scraping & EDA

Collecting, Structuring & Exploring Data for Analysis

PREPARED BY

Name:	Noor Fatima
Student / Employee ID:	CA/DF1/162819
Course / Department:	Data Analytics
Submitted To:	Code Alpha
Date:	20 th June 2026

Tools referenced: Python (BeautifulSoup, Scrapy, pandas, seaborn, SciPy) | Octoparse | ParseHub

TASK 1

Web Scraping: Methodology & Workflow

This report documents the approach, tools, and workflow used to complete the Web Scraping task. It covers library-based scraping with Python, no-code alternatives for non-developers, the process of identifying and collecting relevant public datasets, handling HTML structure and site navigation, and packaging the results into a custom dataset suited to the analysis at hand.

1. Extracting Data with Python Libraries

Two libraries were used to extract data from target websites, chosen based on the complexity and scale of each site:

- **BeautifulSoup** — used for smaller, static pages. It parses HTML/XML and makes it straightforward to search and navigate the document tree (tags, attributes, and text) once the page has been downloaded with a library such as *requests*.
- **Scrapy** — used for larger, multi-page jobs. It is a complete crawling framework with built-in request scheduling, link following, retry logic, and pipelines for cleaning and exporting data, which made it a better fit for crawling many pages across a site such as the example listings platform below.

Example: BeautifulSoup snippet

The snippet below illustrates the pattern used to pull product listings from a (fictional) example retailer, “Northfield Outfitters,” for a pricing-comparison dataset:

```
import requests
from bs4 import BeautifulSoup

url = "https://www.northfieldoutfitters-example.com/catalog"
page = requests.get(url, headers={"User-Agent": "Mozilla/5.0"})
soup = BeautifulSoup(page.text, "html.parser")

for item in soup.select("div.product-card"):
    name = item.select_one(".product-title").get_text(strip=True)
    price = item.select_one(".product-price").get_text(strip=True)
    print(name, price)
```

Example: Scrapy spider

For multi-page crawls, a Scrapy spider was used to follow pagination links automatically across an example job-listings site, “HireWell Boards”:

```
import scrapy
```

```
class HireWellSpider(scrapy.Spider):
    name = "hirewell"
    start_urls = ["https://www.hirewellboards-example.com/jobs"]

    def parse(self, response):
        for job in response.css("li.job-listing"):
            yield {
                "title": job.css(".job-title::text").get(),
                "company": job.css(".job-company::text").get(),
                "location": job.css(".job-location::text").get(),
            }
        next_page = response.css("a.pagination-next::attr(href)").get()
        if next_page:
            yield response.follow(next_page, self.parse)
```

2. Identifying & Collecting Relevant Datasets

Before scraping, each candidate source was screened against the goals of the analysis to avoid collecting noisy or irrelevant data:

- **Define the question first** — e.g. “compare entry-level salaries across three regions” — so that only fields supporting that question are targeted.
- **Check public availability and terms of use** — only pages permitting automated access (per the site’s robots.txt and terms of service) were selected.
- **Assess data freshness and consistency** — pages were checked for regular updates and a stable HTML layout, since frequent redesigns increase scraper maintenance.
- **Sample before scaling** — a handful of pages were scraped manually first to confirm the fields needed (title, price, date, location, etc.) actually existed and were clean.

For this task, the example sources selected were Northfield Outfitters (retail pricing) and HireWell Boards (job postings) — both used here as stand-ins for real public data sources of the same type.

3. No-Code Alternatives for Non-Programmers

For team members without a coding background, point-and-click scraping tools provide a comparable result without writing code. Two were evaluated:

Tool	Strengths	Best Used For
Octoparse	Visual point-and-click workflow builder; cloud scheduling; handles logins and infinite scroll	Recurring scrapes that need to run on a schedule without supervision

ParseHub	Free tier; handles JavaScript-heavy / dynamic pages well; simple desktop app	One-off extractions from modern, script-driven websites
-----------------	--	---

Both tools let the user click the elements to extract directly on the rendered page, generating a structured export (CSV/Excel/JSON) without requiring any Python code — a practical option for quick turnarounds or for teammates handing off requirements to a non-technical analyst.

4. Handling HTML Structure & Web Navigation

Reliable scraping depends on correctly reading a page's structure and handling the ways a site is navigated:

- **Inspecting the DOM** — browser developer tools were used to locate the CSS selectors / XPath for each target element before writing any extraction code.
- **Pagination** — “next page” links and offset-based URLs were followed programmatically (see the Scrapy example above) rather than hard-coding a fixed page count.
- **Dynamic content** — pages that load data via JavaScript after the initial request were identified and handled separately (e.g. via a headless browser or the site's underlying API) instead of being scraped with a basic HTML parser.
- **Rate limiting & politeness** — requests were throttled and a custom User-Agent was set to avoid overloading the target server and to stay within acceptable use limits.
- **Error handling** — missing fields, changed layouts, and failed requests were caught and logged rather than allowed to silently break the dataset.

5. Creating Custom Datasets Tailored to the Analysis

Raw scraped output was cleaned and reshaped into a dataset matching the analysis requirements:

- **Standardizing fields** — prices, dates, and locations were normalized to consistent formats (e.g. currency stripped to numeric, dates to ISO 8601).
- **Deduplication** — repeated listings (common with paginated or refreshed pages) were removed based on a unique key such as product ID or job-posting URL.
- **Schema design** — final columns were chosen to match the analysis question directly, dropping incidental fields picked up during scraping that weren't needed.
- **Export** — the cleaned dataset was saved as CSV for downstream analysis in tools such as Excel, pandas, or a BI dashboard.

Sample output schema

Field	Type	Source
product_name	text	Northfield Outfitters

price_usd	numeric	Northfield Outfitters
job_title	text	HireWell Boards
location	text	HireWell Boards
date_collected	date (ISO 8601)	scrape timestamp

6. Summary

Combining code-based tools (BeautifulSoup for simple pages, Scrapy for larger crawls) with no-code platforms (Octoparse, ParseHub) for non-technical use cases covered the full range of scraping needs for this task. Careful attention to HTML structure, site navigation, and rate-limiting produced a clean, reliable extraction process, and the resulting data was standardized into a custom dataset ready for analysis.

